EOSC 213: Computational Methods in Geological Engineering

Lecture 20: Parameter estimation for ODE models from data

Shadab Ahamed

Department of Earth, Ocean and Atmospheric Sciences,
The University of British Columbia

Tue, Mar 31, 2026

Course logistics

Final exam

- Date: Saturday, April 25, 2026
- Time: 3:30 PM – 6:00 PM
- Venue: UBCV, P. A. Woodward Instructional Resources Centre (IRC), Floor: G, Room: 3

Homework 04

- Release date: by this Friday, 3rd April, 2026
- Submission deadline: ∼17th April, 2026

Reminder

Please check Canvas regularly for updates, announcements, and posting of course materials.

What we will do in Lectures 20, 21, and 22

Until now, we mostly solved the **forward problem**:

- the ODE model is given,
- the parameters are known,
- and we compute the solution numerically.

For the next three classes, we switch to the **inverse problem**:

- data are given,
- some model parameters are unknown,
- and we try to estimate those parameters from the data.

Plan for the next three lectures

Lecture 20: What parameter estimation means for an ODE model; build a loss function and compare model predictions with data.

Lecture 21: Use optimization (gradient descent and PyTorch autograd) to fit unknown parameters automatically.

Lecture 22: Move to a richer nonlinear example with noisy data and discuss practical issues.

Today's goal

Today we will answer the question:

If an ODE model has an unknown parameter, how can we estimate it from the observed data?

By the end of today, you should be able to:

1. identify the unknown parameter(s) in a simple ODE model,
2. simulate the model numerically for a chosen parameter value,
3. compare the simulated solution to observed data,
4. define a **loss function** that measures mismatch.

Main message

Parameter estimation for ODEs means: **choose parameters so that the numerical solution of the model matches the data as well as possible.**

Why is this useful?

In applications, we often know the **form** of the model, but not all of its parameters.

Examples:

- cooling rate in a temperature model,
- growth rate in a population model,
- damping coefficient in an oscillation model,
- decay constant in a contaminant or radioactive decay model.

So the workflow becomes:

$$\text{model} + \text{unknown parameter} + \text{observed data} \longrightarrow \text{estimate the parameter}$$

Important viewpoint

Solving an ODE numerically is not only about prediction. It can also be used inside a data-fitting procedure.

Forward problem vs inverse problem

Forward problem

- ODE is given
- parameter is known
- initial condition is known
- compute the solution

Example:

$$y' = -\lambda y, \qquad y(0) = 10,$$

with $\lambda = 0.25$ known.

Forward problem vs inverse problem

Forward problem

- ODE is given
- parameter is known
- initial condition is known
- compute the solution

Example:

$$y' = -\lambda y, \qquad y(0) = 10,$$

with $\lambda = 0.25$ known.

Inverse problem

- ODE is given
- **some observations are given**
- **parameter is unknown**
- estimate the parameter from the data

Example:

$$y' = -\lambda y, \qquad y(0) = 10,$$

with λ unknown.

Forward problem vs inverse problem

Forward problem

- ODE is given
- parameter is known
- initial condition is known
- compute the solution

Example:

$$y' = -\lambda y, \qquad y(0) = 10,$$

with $\lambda = 0.25$ known.

Inverse problem

- ODE is given
- **some observations are given**
- **parameter is unknown**
- estimate the parameter from the data

Example:

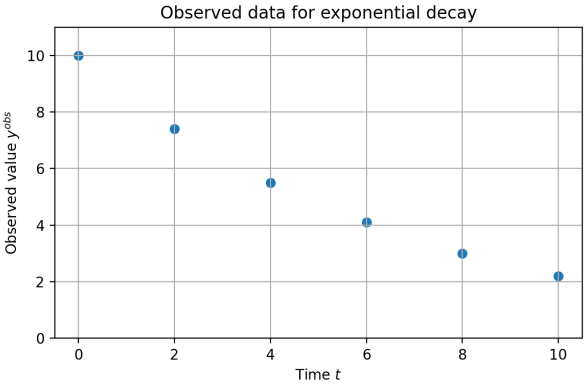
$$y' = -\lambda y, \qquad y(0) = 10,$$

with λ unknown.

Key shift

Earlier in the course, the parameter was an input. Now the parameter becomes an unknown to be inferred from data.

Observed data



Assume we measured the quantity $y(t)$ at the following times:

Time t	Observed value y^{obs}
0	10.0
2	7.4
4	5.5
6	4.1
8	3.0
10	2.2

These are **data points**, not a full formula.

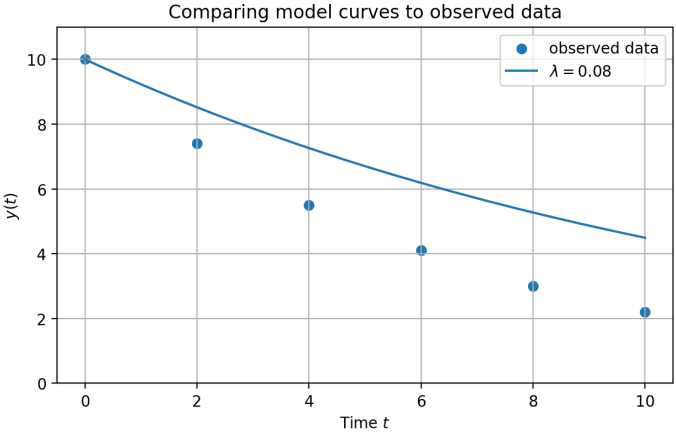
Which value of λ makes the model fit these measurements best?

Trying different values of λ

We keep the model

$$y' = -\lambda y, \qquad y(0) = 10,$$

and compare several candidate values of λ against the observed data.

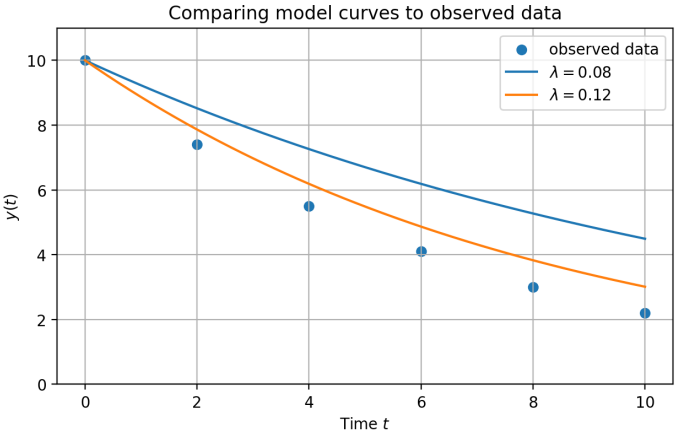


Trying different values of λ

We keep the model

$$y' = -\lambda y, \qquad y(0) = 10,$$

and compare several candidate values of λ against the observed data.

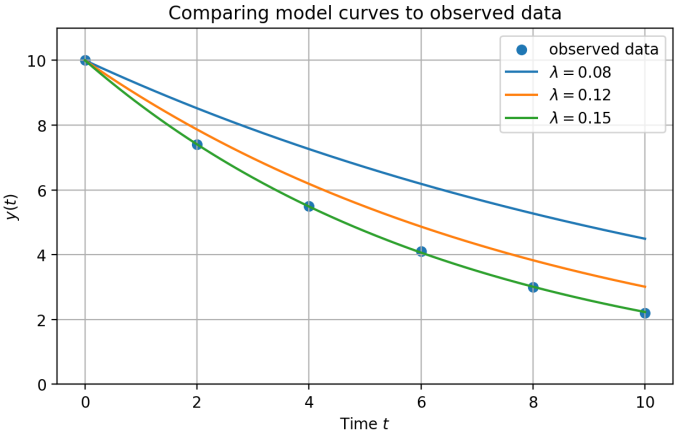


Trying different values of λ

We keep the model

$$y' = -\lambda y, \qquad y(0) = 10,$$

and compare several candidate values of λ against the observed data.

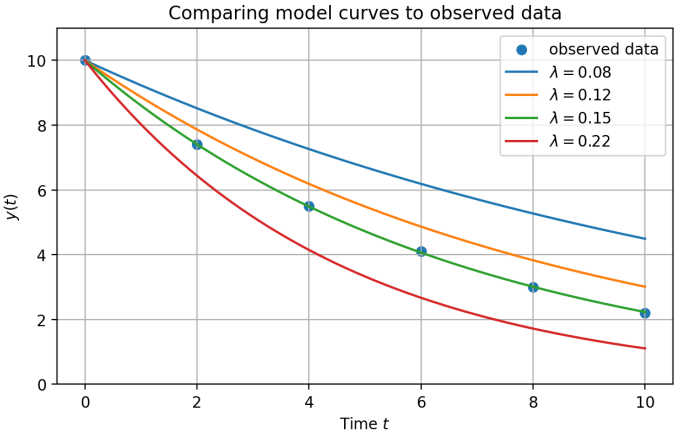


Trying different values of λ

We keep the model

$$y' = -\lambda y, \qquad y(0) = 10,$$

and compare several candidate values of λ against the observed data.



How do we test a guess for λ?

Suppose we guess a value of λ.

Then we can:

1. solve the ODE numerically using that guessed value,
2. compute the model-predicted values at the measurement times,
3. compare those predictions with the observed data.

So one parameter guess gives one predicted curve.

Important connection to the course

Every time we test a parameter value, we must solve an ODE. So the numerical methods you already learned are part of the fitting procedure.

A visual way to think about the task

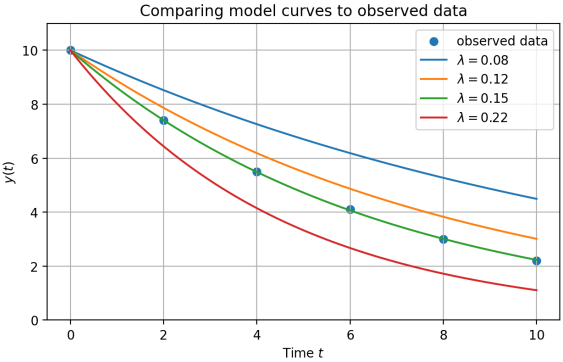
For several different guesses of λ:

- solve the ODE,
- plot the predicted curve,
- compare the curve to the data points.

If λ is too small, the decay is too slow.

If λ is too large, the decay is too fast.

A good value of λ should make the curve pass near the data points.



Numerical model prediction

For a given λ, the ODE is

$$y' = -\lambda y, \qquad y(0) = 10.$$

Using Forward Euler with step size h, we get

$$y_{n+1} = y_n + h(-\lambda y_n) = (1 - h\lambda)y_n.$$

So if λ changes, the whole numerical solution changes.

What changes now?

Earlier, we used a numerical method to approximate a known model solution.

Now we use a numerical method to produce predictions for many possible parameter values.

Residuals: prediction minus observation

At each observation time t_i, we compare:

$$y_\lambda(t_i) \quad \text{(model prediction)} \qquad \text{and} \qquad y_i^{obs} \quad \text{(observed data)}.$$

The difference is called the **residual**:

$$r_i(\lambda) = y_\lambda(t_i) - y_i^{obs}.$$

Interpretation:

- $r_i(\lambda) > 0$: model predicts too high,
- $r_i(\lambda) < 0$: model predicts too low,
- $r_i(\lambda) \approx 0$: model matches well at that point.

A good parameter value should make the residuals small at all data points.

Visualizing the residuals for one chosen value of λ

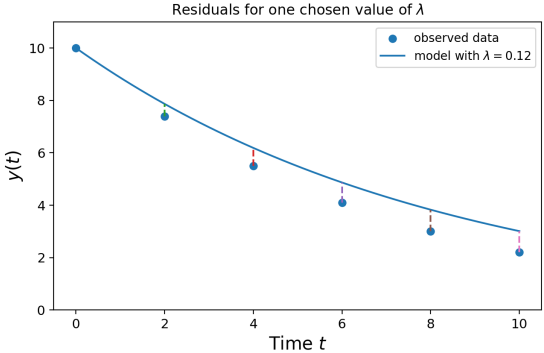
Suppose we choose

$$\lambda = 0.12.$$

Then we solve the model

$$y' = -\lambda y, \qquad y(0) = 10,$$

and compare the predicted curve with the observed data.



At each data point, the vertical dashed line shows the residual:

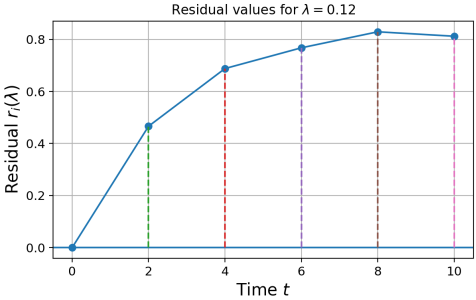
$$r_i(\lambda) = y_\lambda(t_i) - y_i^{obs}.$$

If the model prediction is above the data point, the residual is **positive**. If the model prediction is below the data point, the residual is **negative**.

Residual values for one chosen value of λ

For $\lambda = 0.12$, we can also plot the residuals directly as signed quantities:

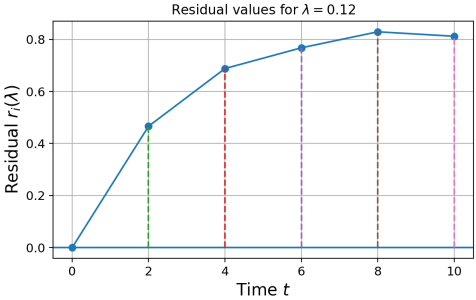
$$r_i(\lambda) = y_\lambda(t_i) - y_i^{obs}.$$



Residual values for one chosen value of λ

For $\lambda = 0.12$, we can also plot the residuals directly as signed quantities:

$$r_i(\lambda) = y_\lambda(t_i) - y_i^{obs}.$$



Interpretation:

- $r_i(\lambda) > 0$: model is above the data,
- $r_i(\lambda) < 0$: model is below the data,
- $r_i(\lambda) = 0$: exact match at that point.

Why this matters

The loss function combines all of these residuals into a single number that measures the overall mismatch between model and data.

Loss function: one number measuring total mismatch

We want a single number that tells us how well a chosen λ fits all the data points.

A standard choice is the **least-squares loss**:

$$L(\lambda) = \frac{1}{N}\sum_{i=1}^{N} r_i(\lambda)^2 = \frac{1}{N}\sum_{i=1}^{N} \left(y_\lambda(t_i) - y_i^{obs}\right)^2.$$

Why square the residuals?

- positive and negative errors do not cancel,
- large errors are penalized more strongly,
- the formula is simple and widely used.

Goal

Find the value of λ that makes $L(\lambda)$ as small as possible.

What does the loss look like?

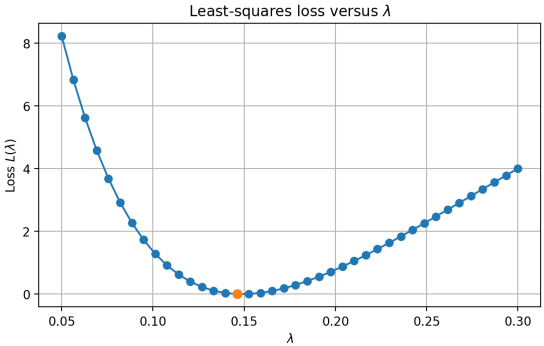
For each guessed value of λ:

1. solve the ODE,
2. compute the residuals,
3. compute the loss $L(\lambda)$.

Then we can make a plot of $L(\lambda)$ versus λ.

Interpretation:

- high loss $\Rightarrow$ poor fit,
- low loss $\Rightarrow$ good fit,
- minimum of the loss $\Rightarrow$ best-fit parameter.

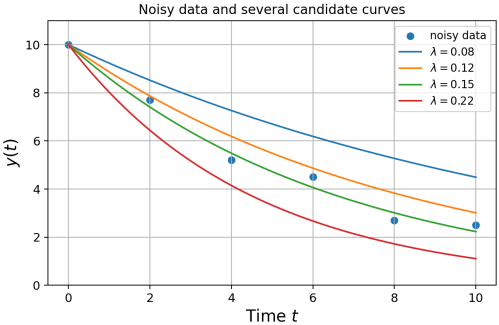


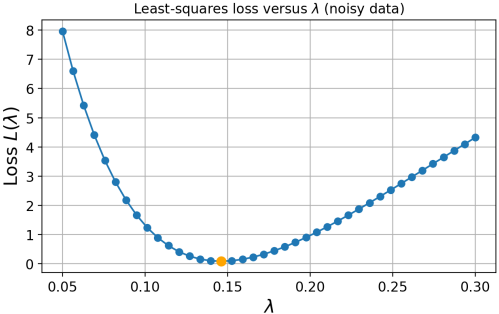
What changes when the data are noisy?

So far, our data were idealized and had essentially no noise.

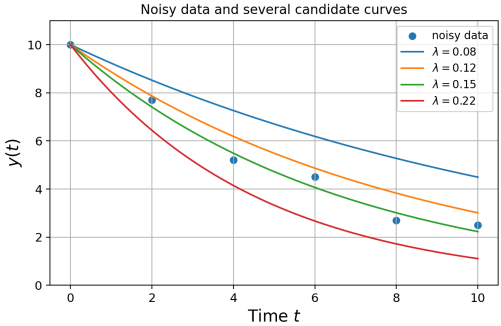
In practice, however, measured data often contain noise:

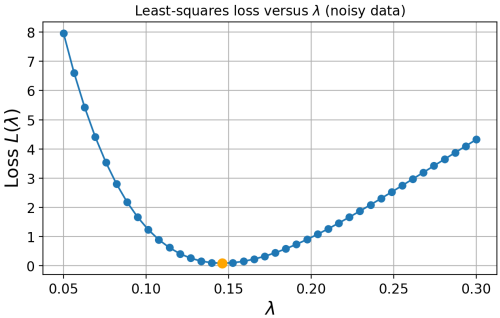
- measurement error,
- instrument error,
- rounding,
- small unmodeled effects.





What changes when the data are noisy?





Important observation

With noisy data, even the best-fit parameter usually does **not** give zero loss. The goal is no longer a perfect match, but rather the **best overall fit**.

Lecture 20 algorithm: scan a range of parameter values

Today, before doing full optimization, we use a simple idea:

1. choose many trial values of λ,
2. for each trial value, solve the ODE numerically,
3. compute the loss,
4. identify which trial value gives the smallest loss.

Lecture 20 algorithm: scan a range of parameter values

Today, before doing full optimization, we use a simple idea:

1. choose many trial values of λ,
2. for each trial value, solve the ODE numerically,
3. compute the loss,
4. identify which trial value gives the smallest loss.

This is not the most efficient method, but it is very instructive.

Lecture 20 algorithm: scan a range of parameter values

Today, before doing full optimization, we use a simple idea:

1. choose many trial values of λ,
2. for each trial value, solve the ODE numerically,
3. compute the loss,
4. identify which trial value gives the smallest loss.

This is not the most efficient method, but it is very instructive.

Why start this way?

It lets us clearly see the connection between

$$\text{parameter} \longrightarrow \text{numerical solution} \longrightarrow \text{residuals} \longrightarrow \text{loss}.$$

PyTorch setup for the observed data

```
import torch

# observed times and data values
t_obs = torch.tensor([0., 2., 4., 6., 8., 10.])
y_obs = torch.tensor([10.0, 7.4, 5.5, 4.1, 3.0, 2.2])

y0 = 10.0
```

These tensors represent the measured data we want to fit.

Pedagogical point

The data are discrete. Our model, however, is continuous in time. Numerical time stepping connects the continuous model to the discrete observations.

PyTorch function to simulate the ODE

```
def forward_euler_decay(lam, h=0.1, t_final=10.0):
    nsteps = int(t_final / h)
    y = torch.tensor(y0)
    t = 0.0

    times = [0.0]
    values = [y]

    for _ in range(nsteps):
        y = y + h * (-lam * y)
        t = t + h
        times.append(t)
        values.append(y)

    return torch.tensor(times), torch.stack(values)
```

Given a chosen value of λ, this function produces a numerical trajectory.

Comparing predictions to the data

```
def loss_for_lambda(lam):
    t_model, y_model = forward_euler_decay(lam)

    # since t_obs = [0,2,4,6,8,10] and h=0.1,
    # the matching indices are [0,20,40,60,80,100]
    idx = torch.tensor([0, 20, 40, 60, 80, 100])
    y_pred = y_model[idx]

    return torch.mean((y_pred - y_obs)**2)
```

This function returns one number: the least-squares mismatch between model and data.

Smaller loss means a better fit.

Scanning many trial values of λ

```
lambda_values = torch.linspace(0.05, 0.30, 100)
loss_values = []

for lam in lambda_values:
    loss_values.append(loss_for_lambda(lam))

loss_values = torch.stack(loss_values)

best_index = torch.argmin(loss_values)
lambda_best = lambda_values[best_index]
```

This gives us the best parameter among the trial values we tested.

Today vs next class

Today we search by scanning many values. In the next lecture, we will learn how optimization methods can update the parameter automatically and more efficiently.

What to notice from this example

1. The ODE model gives a **family of curves**, one for each value of λ.
2. The data select which members of that family are plausible.
3. We need a **numerical solver** to test each parameter value.
4. We need a **loss function** to measure model-data mismatch.
5. Parameter estimation is therefore a combination of:
   - ODE modeling,
   - numerical time stepping,
   - and optimization.

A few important remarks

- Real data are often noisy, so we usually do not expect a perfect match.
- A poor numerical time step can affect the fitted parameter.
- Different parameter values can sometimes produce similar-looking solutions.
- The time interval and the number of observations matter.

This is why the earlier parts of the course matter

Stability, truncation error, and implementation are not just abstract ideas. They affect how reliably we can estimate parameters from data.

Summary of Lecture 20

Today we introduced parameter estimation for ODE models.

Core ideas:

- observed data can be used to infer unknown model parameters,
- for each parameter choice, we solve the ODE numerically,
- residuals measure pointwise mismatch,
- least squares gives one overall loss value,
- best fit means minimizing that loss.

Next lecture

We will move from scanning many trial values to **optimization**: gradient descent, PyTorch autograd, and automatic fitting of ODE parameters.